

Automatisation du Déploiement & renouvellement de Certificats (HTTPS/IIS) PKIs publiques

Certify Management Hub



Certify the web

Fiche d'Action : Déploiement de Certificats vers les Magasins locaux des backends grâce à des tâches PowerShell

Lien de dépôts : <https://github.com/BENELKAID69/Optimedit-projects>
Auteur : Optimed IT (optimedit.fr@gmail.com)

Choix du produit et téléchargement : La suite Certify The Web propose trois solutions distinctes

1. Certify Certificate Manager CCM,
2. Certify Management Hub
3. Certify Management Agent (*il n'existe pas un Certify agent Windows pour IIS*).

Pour mon architecture, il convient de sélectionner le Certify Management Hub (disponible en téléchargement direct à l'adresse <https://certifytheweb.com/home/changelog?product=hub>) plutôt que le CCM. En effet, le Hub permet de piloter centralement la gestion des certificats sur plusieurs serveurs grâce à son API, contrairement au CCM qui se limite à une approche locale.

Ce dépôt contient les outils nécessaires pour automatiser le déploiement sécurisé de certificats SSL depuis un serveur central (API-REST) vers des serveurs web distants ex : (SRV-WEB-02, SRV-WEB-03).

Topologie réseau

Équipement	Rôle	IP	Détail
POST-HP	Poste admin	192.168.1.x	Exécution Postman (client API), RDP
AD-DC	Contrôleur de domaine	192.168.1.x	AD, DNS, DHCP, ADCS – usage pour svc
API-REST	Reverse Proxy / Hub	192.168.1.125	IIS ARR + Certify Management Hub + Scripts PS1
SRV-WEB-02	Backend applicatif 1	192.168.1.33	IIS, sans CCM et sans Certify Agent
SRV-WEB-03	Backend applicatif 2	192.168.1.169	IIS, sans CCM et sans Certify Agent

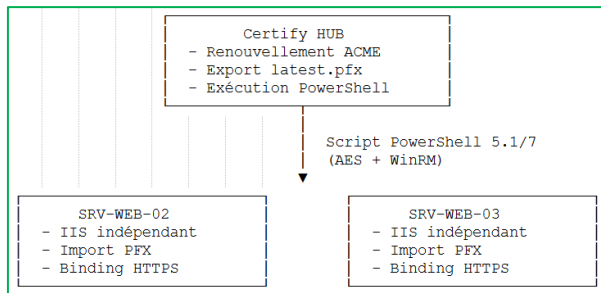
Architecture de la ferme :

Architecture hybride décentralisée à orchestration scriptée :

- ✓ Le HUB est le seul serveur qui renouvelle le certificat via Certify Hub.
- ✓ Les serveurs SRV-WEB-02 et SRV-WEB-03 sont indépendants, chacun avec son propre IIS.
- ✓ Il n'y a aucune configuration centralisée entre les serveurs IIS.
- ✓ Le déploiement du certificat est orchestré par un script PowerShell 5.1 ou 7, pas par Certify.

Ce que l'architecture n'utilise pas :

- ✓ IIS Shared Configuration
- ✓ IIS Central Certificate Store CCS
- ✓ Certify Agent installé sur chaque serveur backend (non disponible pour Windows)



Objectif

Optimisation et sécurisation de la ferme de serveurs IIS backends en deux phases. Initialement articulée autour d'un modèle de **SSL Offloading** (terminaison TLS au niveau du reverse-proxy) géré automatiquement par le « **Certify Management Hub** », l'architecture sera évoluée pour implémenter un chiffrement **End-to-End TLS**.

Cette évolution garantit que le flux de données reste chiffré de bout en bout, du client final jusqu'aux nœuds IIS backends.

La maintenance de ce niveau de sécurité est assurée par une **automatisation complète** (via deux tâches planifiées et scripts PowerShell personnalisés, garantissant le déploiement massif et le renouvellement sans intervention humaine des certificats sur l'ensemble du parc de serveurs.

La partie de configuration d'automatisation est configurée par deux versions, en premier, avec Windows PowerShell version 5.1 puis avec une seconde version configurée avec PowerShell version 7.x «pswh.exe», les deux versions sont validées avec des outils natifs Windows et requêtes AP Postman.

Phase 1 : **SSL Offloading**

[Client] ---- HTTPS ----> [API-REST:443] ---- HTTP ----> [SRV-WEB-02/03:80]

(ARR + Hub) (certificat api-rest.optimedit.eu seulement sur le revers-proxy ARR)

Phase 2 : **End-to-End TLS**

[Client] ---- HTTPS ----> [API-REST:443] ---- HTTPS ----> [SRV-WEB-02/03:443]

(ARR + Hub) (même certificat api-rest.optimedit.eu, propagé automatiquement sur les serveurs backends)

Architecture de Sécurité

Pour garantir la robustesse du service de déploiement, ces scripts utilisent :

- **Chiffrement AES 256 bits** : Indépendant du profil utilisateur, ce qui le rend idéal pour un service tournant en tâche de fond (compte SYSTEM).
- **Contrôle d'accès strict (ACL)** : Seuls le compte SYSTEM et le groupe Administrateurs ont accès aux secrets, empêchant toute lecture non autorisée.

Structure des dossiers

1. Pour tâche de l'export PFX

- **C:\CertifyScripts** : Dossier racine.
 - **\export** : Contient le certificat exporté (latest.pfx) prêt à être déployé.

2. Pour tâche du script Windows PowerShell version 5.1 (powershell.exe)

- **C:\CertifyScripts** : Dossier racine.
 - **\secure** : Contient la clé AES (aes.key) et les secrets chiffrés (.enc).
 - **\logs** : Contient l'historique des déploiements.

3. Pour tâche du script PowerShell version 7 (pwsh.exe)

- **C:\CertifyScripts** : Dossier racine.
 - **\secure_pwsh** : Contient la clé AES (aes.key) et les secrets chiffrés (.enc).
 - **\logs_pwsh** : Contient l'historique des déploiements.

Ordre d'exécution et Procédure – version PowerShell version 5.1

Étape 1 : Initialisation (Une seule fois)

Exécutez 01.Gen_AES-Key.ps1 en mode Administrateur.

Ce script génère une clé unique et verrouille son accès via des SID (SYSTEM/Administrateurs) pour éviter toute erreur de résolution de nom.

Étape 2 : Configuration des secrets (Ponctuel)

1. Modifiez 02.ConvertTo-SecureString-aes.ps1 avec vos mots de passe réels (Password_SVC_Ici et Password_PFX_Ici).
2. Exécutez le script. Cela génère les fichiers deploy-pwd.enc et pfx-pwd.enc.
3. **IMPORTANT** : Supprimez immédiatement 02.ConvertTo-SecureString-aes.ps1 après usage, car il contient les mots de passe en clair.

Étape 3 : Sécurisation finale (Ponctuel)

Exécutez 03.secure-enc-file.ps1.

Ce script applique les mêmes restrictions d'accès strictes (ACL) aux fichiers .enc générés à l'étape précédente.

Note : pour la version PowerShell 7 (pwsh), ces trois scripts ont été fusionnés sur un seul, les deux mots de passe ne sont pas enregistrés sur le script mais demandés durant l'exécution de script :

```
$SecurePwd = Read-Host "Mot de passe svc-certdeploy" -AsSecureString  
$SecurePfxPwd = Read-Host "Mot de passe du PFX" -AsSecureString
```

Étape 4 : Déploiement automatisé (deux tâches planifiées + PS1)

Création deux tâches planifiées sur le Hub, une tâche pour déploiement et renouvellement certificats automatiquement, appelée «Deploy-cert-to-backends» et une seconde pour exporter le certificat dans un fichier format « .PFX » , appelée «latest.pfx » ,

Configuration de la Deployment Task dans le Hub

Dans l'UI du Hub, sur le certificat `api-rest.optimedit.eu` → Tasks → **Add Task** (dans *Deployment Task* et non dans *Pre-Request Task*):

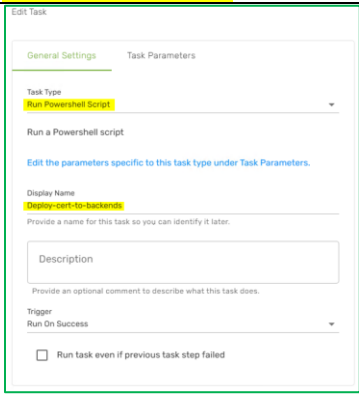
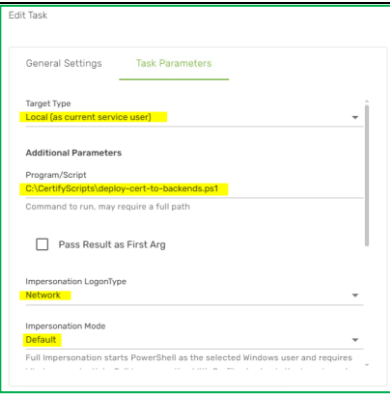
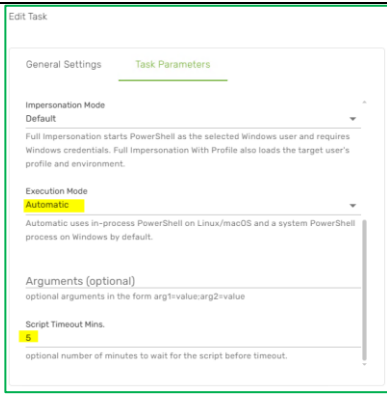
01. Tâche Deploy-cert-to-backends

Champ	Valeur
Task Type	Run Powershell Script
Task Name / Display Name	Deploy-cert-to-backends
Program/Script	C:\CertifyScripts\ deploy-cert-to-backends.ps1
Pass Result as First Arg	✗ Décoché
Impersonation LogonType	Network
Impersonation Mode	Default
Execution Mode	Automatic (par défaut)
Arguments (optional)	(laisser vide — plus aucun secret à transmettre, tous les paramètres ont des valeurs par défaut dans le script)
Script Timeout Mins.	5 (raisonnable pour pas beaucoup serveurs)
Run task even if previous task step failed	✗ Décoché
Position	Après la tâche CertificateExport existante (postRequestTask, 2ème position)

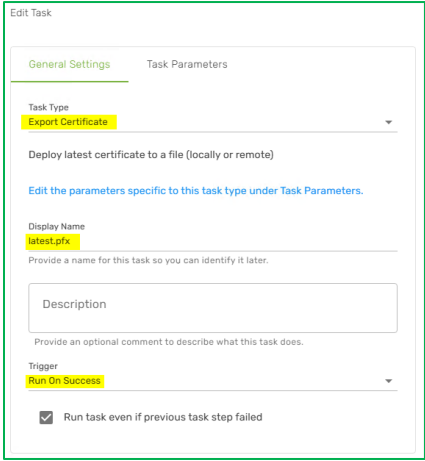
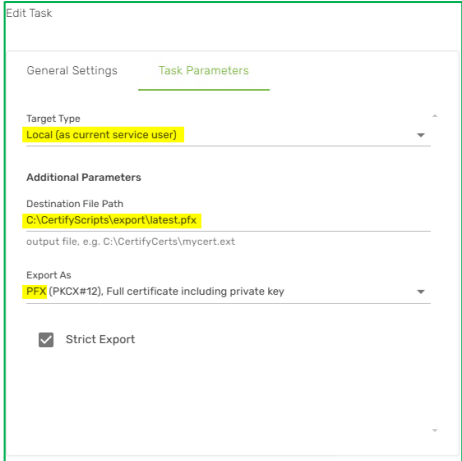
02. Tâche latest.pfx : une seconde tâche d'export de certificat au format .PFX vers « C:\CertifyScripts\export », choisir Task type « Export Certificat » et Export As PFX.

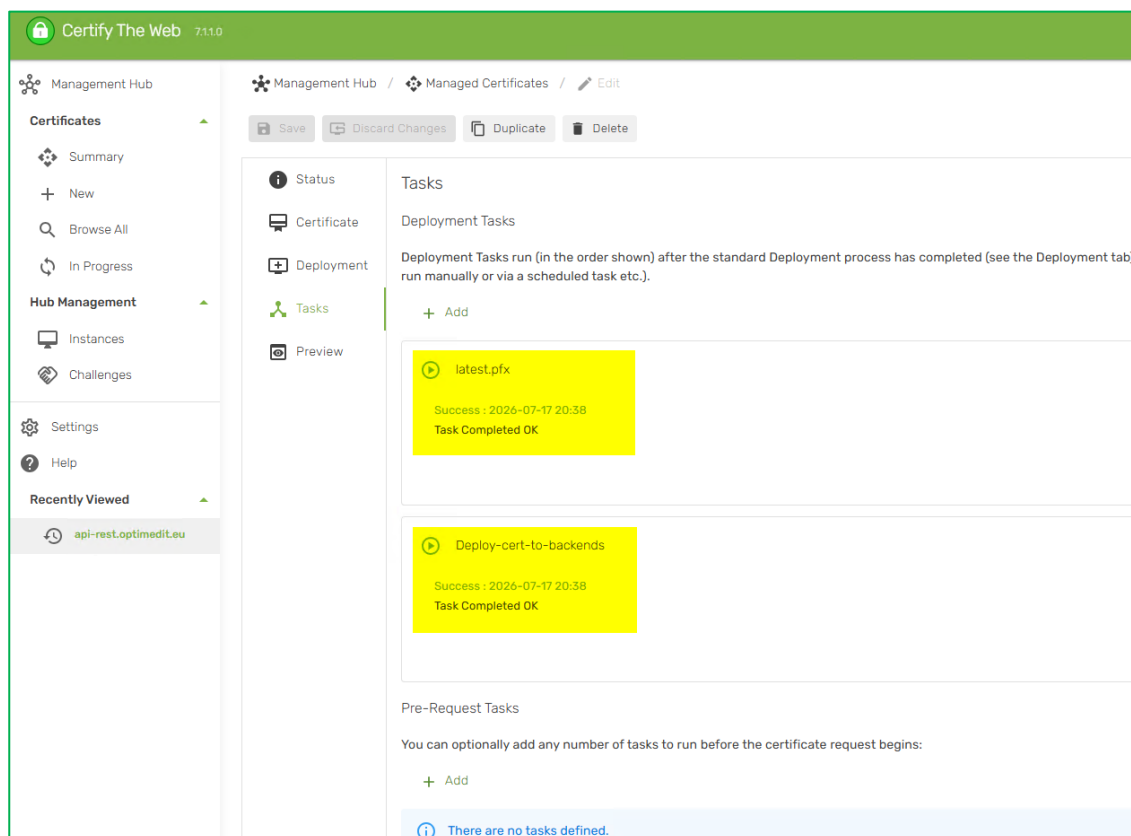
Tâche Deploy-cert-to-backends (provider API = « Certify.Providers.DeploymentTasks.**Powershell** »)

Cette tâche créée avec un script Windows PowerShell version 5.1 – voir plus loin une autre tâche avec un script PowerShell Core pwsh.exe version 7.x, provider API « ShellExecute »

		
---	---	---

Tâche latest.pfx (provider API = « Certify.Providers.DeploymentTasks.CertificateExport »))

	
---	--



Le script **deploy-cert-to-backends.ps1** est appelé automatiquement par le **Certify Management Hub**.

- Il déchiffre les secrets à la volée grâce à aes.key.
- Il ouvre une session WinRM vers les serveurs cibles.
- Il installe le certificat, met à jour les bindings IIS et nettoie les anciens certificats.
- Toutes les actions sont tracées dans le dossier \logs\.

Résumé des scripts

Script	Rôle	Statut après usage
01.Gen_AES-Key.ps1	Génération de la clé AES et application des ACL.	Conserver
02.ConvertTo-SecureString-aes.ps1	Chiffrement initial des mots de passe.	Supprimer
03.secure-enc-file.ps1	Sécurisation ACL des fichiers .enc.	Conserver
deploy-cert-to-backends.ps1	Moteur de déploiement IIS via WinRM.	Conserver

Note de sécurité : Une fois le fichier .enc et aes.key sécurisés, le script contenant les mots de passe en clair (02) ne doit plus exister sur le système pour éviter toute fuite d'information. Seul le script de déploiement final doit rester présent.

Validation de configuration des deux tâches palnifiées du renouvellement automatique du certificat TLS (logs + requêtes Postman)

```
2026-07-17 22:38 [INFO] === D  PLOIEMENT TERMINE SUR TOUS LES SERVEURS ===
2026-07-17 22:38 [INFO] Log complet disponible : C:\CertifyScripts\logs\deploy_20260717_203828.log
2026-07-17 20:38 [INF] latest.pfx :: Task Completed OK
2026-07-17 20:38 [INF] Deploy-cert-to-backends :: Task Completed OK
```

Variables :

hub_api_url = https://api-rest.optimedite.eu:8443
access_token = sortie de requ  te 'POST {{hub_api_url}}/api/v1/auth/login' = "accessToken": "eyJ...DIU"
instanceld = 17dfbdd7-b834-4da0-8f3c-e18f8395b48a
managedCertId = f9712ad8-bd49-430c-8cff-370633369e53
format = pfx

Requ  te

GET {{hub_api_url}}/internal/v1/certificate/{{instanceld}}/settings/{{managedCertId}}

The image displays four screenshots of the Postman API client, arranged in a 2x2 grid, showing the configuration and results of GET requests to the Optimedite API.

- Top Left:** A GET request to `[[hub_api_url]]/internal/v1/certificate/[[instanceld]]/settings/[[managedCertId]]`. The response is a JSON object (200 OK) containing task configuration details such as `taskName: "latest.pfx"`, `taskType: "CertificateExport"`, and `lastResult: "Task Completed OK"`.
- Top Right:** A GET request to the same endpoint. The response is a JSON object (200 OK) containing task configuration details such as `taskName: "Deploy-cert-to-backends"`, `taskType: "Certify.Providers.DeploymentTasks.Powershell"`, and `lastResult: "Task Completed OK"`.
- Bottom Left:** A GET request to the same endpoint. The response is a JSON object (200 OK) containing task configuration details such as `taskName: "latest.pfx"`, `taskType: "CertificateExport"`, and `lastResult: "Task Completed OK"`.
- Bottom Right:** A GET request to the same endpoint. The response is a JSON object (200 OK) containing task configuration details such as `taskName: "latest.pfx"`, `taskType: "CertificateExport"`, and `lastResult: "Task Completed OK"`.

Vérification de la propagation « End-to-End TLS »

Récupérer le thumbprint

A. Avec PowerShell (URLs testés sur internet et en local) :

Avec enregistrement A OVH	Sans enregistrement A OVH
<pre>PS C:\Windows\System32> test-NetConnection srv-web-02.optimedit.eu -Port 443 ComputerName : srv-web-02.optimedit.eu RemoteAddress : 78.118.220.225 RemotePort : 443 InterfaceAlias : Ethernet SourceAddress : 192.168.1.225 TcpTestSucceeded : True PS C:\Windows\System32> test-NetConnection srv-web-03.optimedit.eu -Port 443 ComputerName : srv-web-03.optimedit.eu RemoteAddress : 78.118.220.225 RemotePort : 443 InterfaceAlias : Ethernet SourceAddress : 192.168.1.225 TcpTestSucceeded : True</pre>	<pre>PS C:\Users\Driss> test-NetConnection -ComputerName srv-web-02.optimedit.eu -Port 443 ComputerName : srv-web-02.optimedit.eu RemoteAddress : 192.168.1.33 RemotePort : 443 InterfaceAlias : Ethernet SourceAddress : 192.168.1.225 TcpTestSucceeded : True PS C:\Users\Driss> test-NetConnection -ComputerName srv-web-03.optimedit.eu -Port 443 ComputerName : srv-web-03.optimedit.eu RemoteAddress : 192.168.1.169 RemotePort : 443 InterfaceAlias : Ethernet SourceAddress : 192.168.1.225 TcpTestSucceeded : True PS C:\Users\Driss> </pre>

```
Get-thumbprint.ps1 X
1 $urls = @("https://srv-web-02.optimedit.eu", "https://srv-web-03.optimedit.eu", "https://api-rest.optimedit.eu")
2
3 foreach ($url in $urls) {
4     try {
5         $request = [System.Net.HttpWebRequest]::Create($url)
6         $request.ServerCertificateValidationCallback = { $true } # Ignore les erreurs de chaîne de confiance pour la lecture seule
7         $request.GetResponse() | Out-Null
8         $cert = $request.ServicePoint.Certificate
9
10        Write-Host "URL : $url" -ForegroundColor Cyan
11        Write-Host "Thumbprint : $($cert.GetCertHashString())" -ForegroundColor Green
12        Write-Host "-----"
13    } catch {
14        Write-Host "Échec de connexion à $url : $($_.Exception.Message)" -ForegroundColor Red
15    }
16 }
17 }
```

```
URL : https://srv-web-02.optimedit.eu
Thumbprint : 000A8BA353159D2CD434186F3BC641B7C787D173
-----
URL : https://srv-web-03.optimedit.eu
Thumbprint : 000A8BA353159D2CD434186F3BC641B7C787D173
-----
URL : https://api-rest.optimedit.eu
Thumbprint : 000A8BA353159D2CD434186F3BC641B7C787D173
-----
PS C:\Users\Driss>
```

B. Avec un client API comme Postman ou client API sur navigateur Web :

On constate les mêmes hash de certificat - **000A8BA353159D2CD434186F3BC641B7C787D173** - visibles sur les réponses des requêtes GET HTTPS des deux serveurs, identique à celui du revers-proxy.

GEThttps://{{srv-web-02.optimedit.eu}}

DocsParamsAuthorizationHeaders6BodyScriptsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQL

This request does not have a body

BodyCookiesHeaders8Test Results

HTMLPreviewVisualize

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
2 <html xmlns="http://www.w3.org/1999/xhtml">
3
4 <head>
5   <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
6   <title>IIS Windows Server</title>
7   <h1>C'EST LE SERVEUR 02</h1>
8   <style type="text/css">
9     <!--
10     body {
11       color: #000000;
```

ConsoleTerminalIssues

```
validTo: "Oct 15 17:39:52 2026 GMT"
fingerprint: "00:0A:8B:A3:53:15:90:2C:04:34:18:6F:3B:C6:41:87:C7:07:D1:73"
serialNumber: "06f9b7ca8e279ea5027de9df0941dd7b7569"
Request Headers
Authorization: "Bearer *****"
User-Agent: "PostmanRuntime/7.54.0"
Accept: "*/*"
Postman-Token: "38df7396-c657-4d2b-a6e2-6b58b76358ab"
Accept-Encoding: "gzip, deflate, br"
:path: "/"
:method: "GET"
:authority: "192.168.1.33"
:scheme: "https"
Response Headers
:status: 200
content-type: "text/html"
last-modified: "Sun, 06 Jul 2026 18:51:39 GMT"
accept-ranges: "bytes"
etag: "ab25a50efcd1:0"
server: "Microsoft-IIS/10.0"
date: "Sun, 19 Jul 2026 17:11:25 GMT"
content-length: "733"
```

GEThttps://{{srv-web-03.optimedit.eu}}

DocsParamsAuthorizationHeaders9BodyScriptsSettings

Bearer ...Token

{{refresh_token}}

BodyCookiesHeaders8Test Results

HTMLPreviewVisualize

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
2 <html xmlns="http://www.w3.org/1999/xhtml">
3
4 <head>
5   <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
6   <title>IIS Windows Server</title>
7   <h1>C'EST LE SERVEUR 03</h1>
8   <style type="text/css">
9     <!--
10     body {
11       color: #000000;
12       background-color: #0072C6;
```

ConsoleTerminalIssues

```
> subject: {}
> issuer: {}
validFrom: "Jul 17 17:39:53 2026 GMT"
validTo: "Oct 15 17:39:52 2026 GMT"
fingerprint: "00:0A:8B:A3:53:15:90:2C:04:34:18:6F:3B:C6:41:87:C7:07:D1:73"
serialNumber: "06f9b7ca8e279ea5027de9df0941dd7b7569"
Request Headers
Content-Type: "application/json"
Authorization: "Bearer phacV6KpD3JfOPD33L7LNl3vLaaCwF/ZaATm9R2Hb1s="
User-Agent: "PostmanRuntime/7.54.0"
Accept: "*/*"
Postman-Token: "b9eee09f-71d0-481a-9b42-54e121b15b67"
Accept-Encoding: "gzip, deflate, br"
Content-Length: "70"
:path: "/"
:method: "GET"
:authority: "192.168.1.169"
:scheme: "https"
Request Body
Response Headers
:status: 200
```


Amélioration de la tâche de déploiement avec script PowerShell v7 (pwsh.exe)

Champ	Valeur
Task Type	Run... (provider interne : ShellExecute)
Task Name	Deploy-cert-to-backends-pwsh
Program/Script	C:\Program Files\PowerShell\7\pwsh.exe
Arguments	-File "C:\CertifyScripts\scripts_pwsh\deploy-cert-to-backends-pwsh-7.ps1"
Launch New Process	false
Impersonation	Default
LogonType	Network
Timeout	5 minutes
Position	Après CertificateExport

Tâche Deploy-cert-to-backends-pwsh (provider API = « Certify.Providers.DeploymentTasks.ShellExecute »)

Edit Task

General Settings
Task Parameters

Task Type
Run...

Run a program, batch file or custom script
[Edit the parameters specific to this task type under Task Parameters.](#)

Display Name
Deploy-cert-to-backends-pwsh
Provide a name for this task so you can identify it later.

Description

Trigger
Run On Success

☐ Run task even if previous task step failed

Edit Task

General Settings
Task Parameters

Target Type
Local (as current service user)

Additional Parameters

Program/Script
C:\Program Files\PowerShell\7\pwsh.exe
Command to run, may require a full path. On Linux, this must be an executable script (e.g. with shebang).

Arguments (optional)
-File "C:\CertifyScripts\pwsh\deploy-cert-to-backends-pwsh-7.ps1"

Script Timeout Mins.
5
optional number of minutes to wait for the script before timeout.

☐ Launch New Process

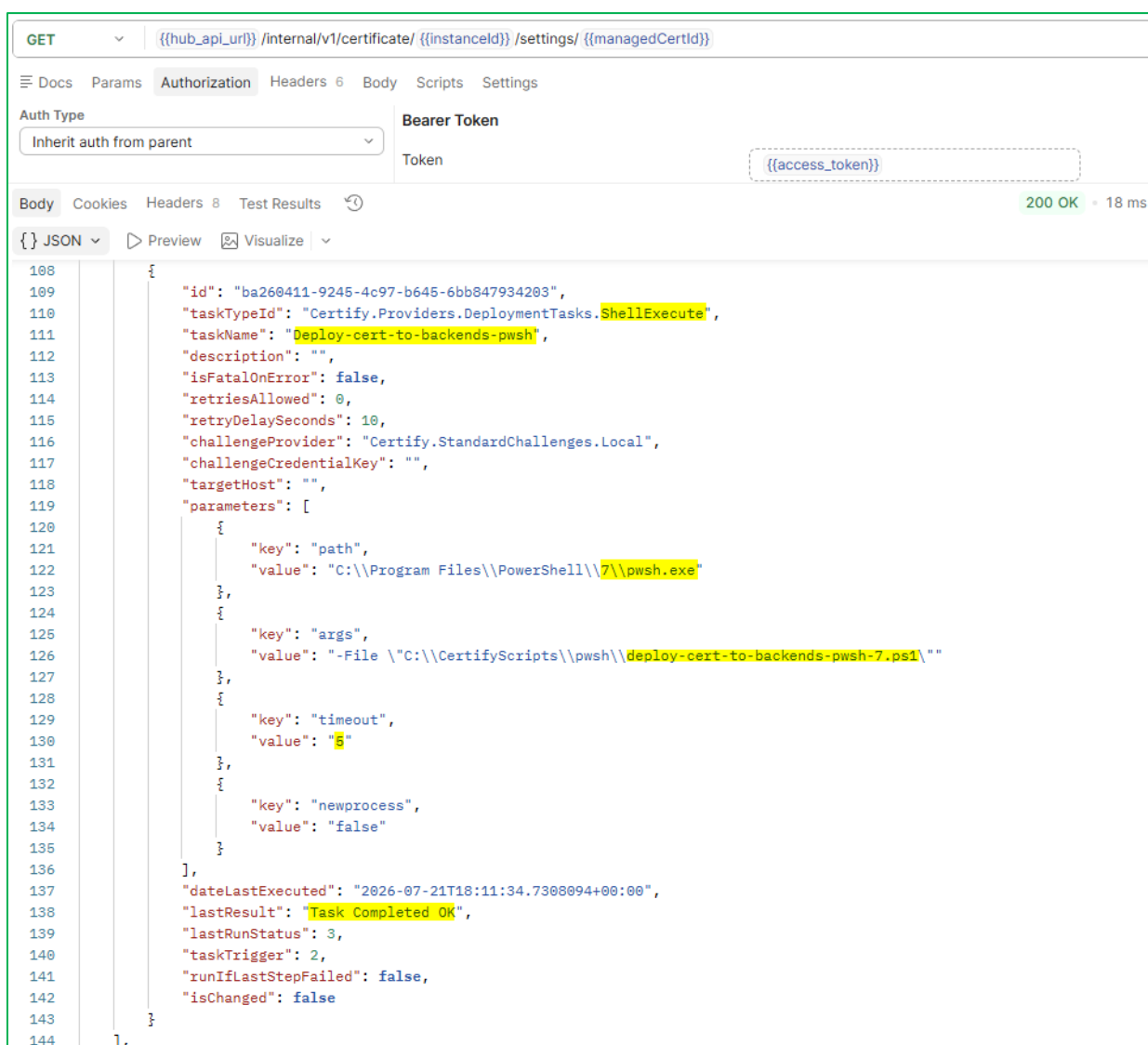
```

2026-07-21 20:11 [INF] Version PowerShell utilis,e : 7.6.3
2026-07-21 22:11 [INFO] === DBUT DU DPLOIEMENT ===
2026-07-21 22:11 [INFO] Chargement de la cl, AES...
2026-07-21 22:11 [INFO] Chargement des secrets chiff,r,s...
2026-07-21 22:11 [INFO] Secrets charg,s avec succs.
2026-07-21 22:11 [INFO] === D,ploiement vers SRV-WEB-02 ===
2026-07-21 22:11 [INFO] Ouverture de la session distante...
2026-07-21 22:11 [INFO] Session ouverte.
2026-07-21 22:11 [INFO] Suppression ancien PFX temporaire...
2026-07-21 22:11 [INFO] Copie du nouveau PFX...
2026-07-21 22:11 [INFO] Copie OK.
2026-07-21 22:11 [INFO] Import du certificat + mise .. jour IIS...
2026-07-21 22:11 [INFO] Binding mis .. jour avec thumbprint : Import du certificat... Certificat import, : 2AE7E408DB28922182725D062CBA7043AD83FA73
2026-07-21 22:11 [INFO] Session ferm,e.
2026-07-21 22:11 [INFO] === D,ploiement vers SRV-WEB-03 ===
2026-07-21 22:11 [INFO] Ouverture de la session distante...
2026-07-21 22:11 [INFO] Session ouverte.
2026-07-21 22:11 [INFO] Suppression ancien PFX temporaire...
2026-07-21 22:11 [INFO] Copie du nouveau PFX...
2026-07-21 22:11 [INFO] Copie OK.
2026-07-21 22:11 [INFO] Import du certificat + mise .. jour IIS...
2026-07-21 22:11 [INFO] Binding mis .. jour avec thumbprint : Import du certificat... Certificat import, : 2AE7E408DB28922182725D062CBA7043AD83FA73
2026-07-21 22:11 [INFO] Session ferm,e.
2026-07-21 22:11 [INFO] === DPLOIEMENT TERMIN SUR TOUS LES SERVEURS ===
2026-07-21 22:11 [INFO] Log complet disponible : C:\CertifyScripts\logs_pwsh\deploy_20260721_201135.log
2026-07-21 20:11 [INF] latest.pfx :: Task Completed OK
2026-07-21 20:11 [INF] Deploy-cert-to-backends-pwsh :: Task Completed OK

```

```
Get-thumbprints1 X
1 $urls = @("https://srv-web-02.optimedit.eu", "https://srv-web-03.optimedit.eu", "https://api-rest.optimedit.eu")
2
3 foreach ($url in $urls) {
4     try {
5         $request = [System.Net.HttpWebRequest]::Create($url)
6         $request.ServerCertificateValidationCallback = { $true } # Ignore les erreurs de chaîne de confiance pour la lecture seule
7         $response = $request.GetResponse() | Out-Null
8         $cert = $request.ServicePoint.Certificate
9
10        Write-Host "URL : $url" -ForegroundColor Cyan
11        Write-Host "Thumbprint : $($cert.GetCertHashString())" -ForegroundColor Green
12        Write-Host "-----"
13    } catch {
14        Write-Host "Échec de connexion à $url : $($_.Exception.Message)" -ForegroundColor Red
15    }
16 }
17
18 } catch {
19     Write-Host "Échec de connexion à $url : $($_.Exception.Message)" -ForegroundColor Red
20 }
21
22 URL : https://srv-web-02.optimedit.eu
23 Thumbprint : 2AE7E408DB28922182725D062CBA7043AD83FA73
24 -----
25 URL : https://srv-web-03.optimedit.eu
26 Thumbprint : 2AE7E408DB28922182725D062CBA7043AD83FA73
27 -----
28 URL : https://api-rest.optimedit.eu
29 Thumbprint : 2AE7E408DB28922182725D062CBA7043AD83FA73
30 -----
31
32 PS C:\Users\Driss>
```

Visualiser la configuration avec une requête d'un client API (Postman)



```
GET {{hub_api_url}} /internal/v1/certificate/ {{instanceId}} /settings/ {{managedCertId}}

Auth Type: Inherit auth from parent
Bearer Token: {{access_token}}

Body: JSON
200 OK - 18 ms

{
  "id": "ba260411-9245-4c97-b645-6bb847934203",
  "taskId": "Certify.Providers.DeploymentTasks.ShellExecute",
  "taskName": "Deploy-cert-to-backends-pwsh",
  "description": "",
  "isFatalOnError": false,
  "retriesAllowed": 0,
  "retryDelaySeconds": 10,
  "challengeProvider": "Certify.StandardChallenges.Local",
  "challengeCredentialKey": "",
  "targetHost": "",
  "parameters": [
    {
      "key": "path",
      "value": "C:\\Program Files\\PowerShell\\7\\pwsh.exe"
    },
    {
      "key": "args",
      "value": "-File \"C:\\CertifyScripts\\pwsh\\deploy-cert-to-backends-pwsh-7.ps1\""
    },
    {
      "key": "timeout",
      "value": "5"
    },
    {
      "key": "newprocess",
      "value": "false"
    }
  ],
  "dateLastExecuted": "2026-07-21T18:11:34.7308094+00:00",
  "lastResult": "Task Completed OK",
  "lastRunStatus": 3,
  "taskTrigger": 2,
  "runIfLastStepFailed": false,
  "isChanged": false
}
```

On voit que la tâche « Deploy-cert-to-backends-pwsh » a été configurée grâce au provider « Certify.Providers.DeploymentTasks.ShellExecute » et non de celui de PowerShell « Certify.Providers.DeploymentTasks.Powershell » comme la tâche précédente « Deploy-cert-to-backends ».

Statuts Certify Hub

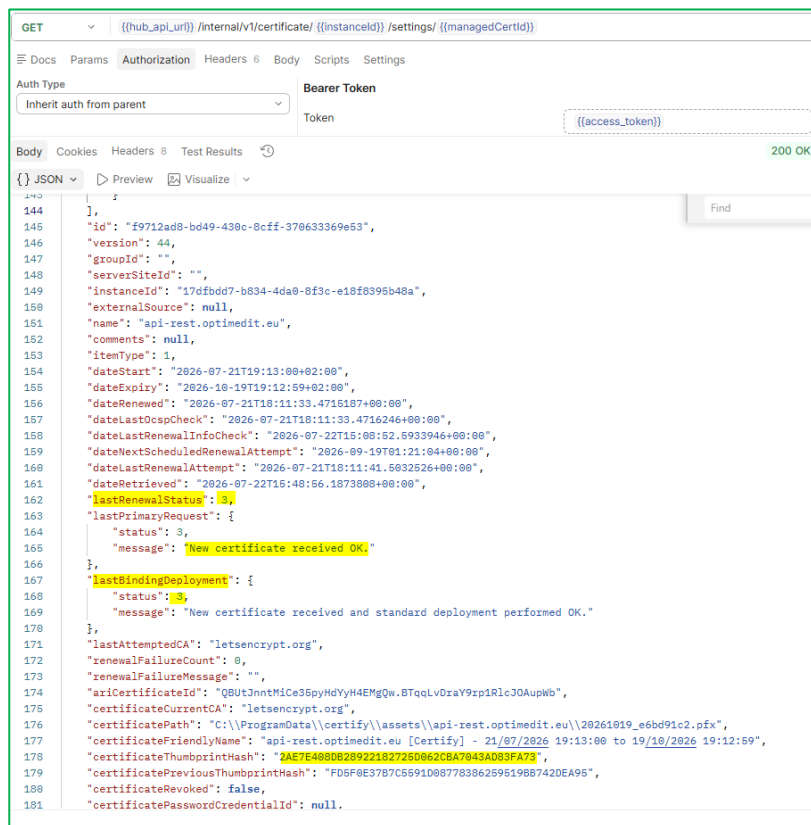


Tableau d'interprétation des statuts Certify (ces valeurs concernent seulement le renouvellement OK sur le HUB)

Champ	Valeur	Signification	Interprétation opérationnelle
lastRenewalStatus	3	Succès	Le renouvellement ACME/Let's Encrypt s'est déroulé correctement sur le HUB.
lastPrimaryRequest.status	3	Succès	Le certificat a été reçu du CA (Let's Encrypt) sans erreur.
lastPrimaryRequest.message	<i>New certificate received OK.</i>	Message explicatif	Le HUB a bien récupéré le nouveau certificat.
lastBindingDeployment.status	3		

Pour les serveurs distants ?

C'est les deux scripts qui gèrent les serveurs backend (SRV-WEB-02, SRV-WEB-03)

Et c'est les logs de scripts qui disent le succès du déploiement multi-serveurs est dans (logs ou logs_pwsh) :

```

=== Déploiement vers SRV-WEB-02 ===
Session ouverte.
Binding mis à jour avec thumbprint : 2AE7E408DB28922182725D062CBA7043AD83FA73
Session fermée.
=== Déploiement vers SRV-WEB-03 ===
Session ouverte.
Binding mis à jour avec thumbprint : 2AE7E408DB28922182725D062CBA7043AD83FA73
Session fermée.
=== DÉPLOIEMENT TERMINÉ SUR TOUS LES SERVEURS ===
  
```